Skill-08

```
 GNU nano 8.7.1                                  Skill8.c *
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

#define MAX_VARS 20
#define MAX_LEN 200

struct Variable {
    char name[50];
    char value[MAX_LEN];
};

struct Variable vars[MAX_VARS];
int var_count = 0;

/* Find variable */
int find_variable(char *name)
{
    for (int i = 0; i < var_count; i++)
    {
        if (strcmp(vars[i].name, name) == 0)
            return i;
    }

    return -1;
}

/* Set or update variable */
void set_variable(char *name, char *value)
{
    int index = find_variable(name);

    if (index == -1)
    {
        strcpy(vars[var_count].name, name);
        strcpy(vars[var_count].value, value);
        var_count++;
    }
    else
    {
```

```
 GNU nano 8.7.1                                  Skill8.c *
    {
        strcpy(vars[index].value, value);
    }
}

/* Get variable value */
char *get_variable(char *name)
{
    int index = find_variable(name);

    if (index == -1)
        return NULL;

    return vars[index].value;
}

/* Expand variables recursively */
void expand_text(char *input, char *output, int depth)
{
    if (depth > 5)
    {
        strcpy(output, "[MAX NESTING]");
        return;
    }

    char temp[MAX_LEN] = "";
    int i = 0;

    while (input[i] != '\0')
    {
        if (input[i] == '$')
        {
            i++;

            char name[50];
            int j = 0;

            while (input[i] != '\0' &&
                   ((input[i] >= 'A' && input[i] <= 'Z') ||
                    (input[i] >= 'a' && input[i] <= 'z') ||
```



```
 GNU nano 8.7.1                                  Skill8.c *
                input[i] == '_'))
            {
                name[j++] = input[i++];
            }

            name[j] = '\0';

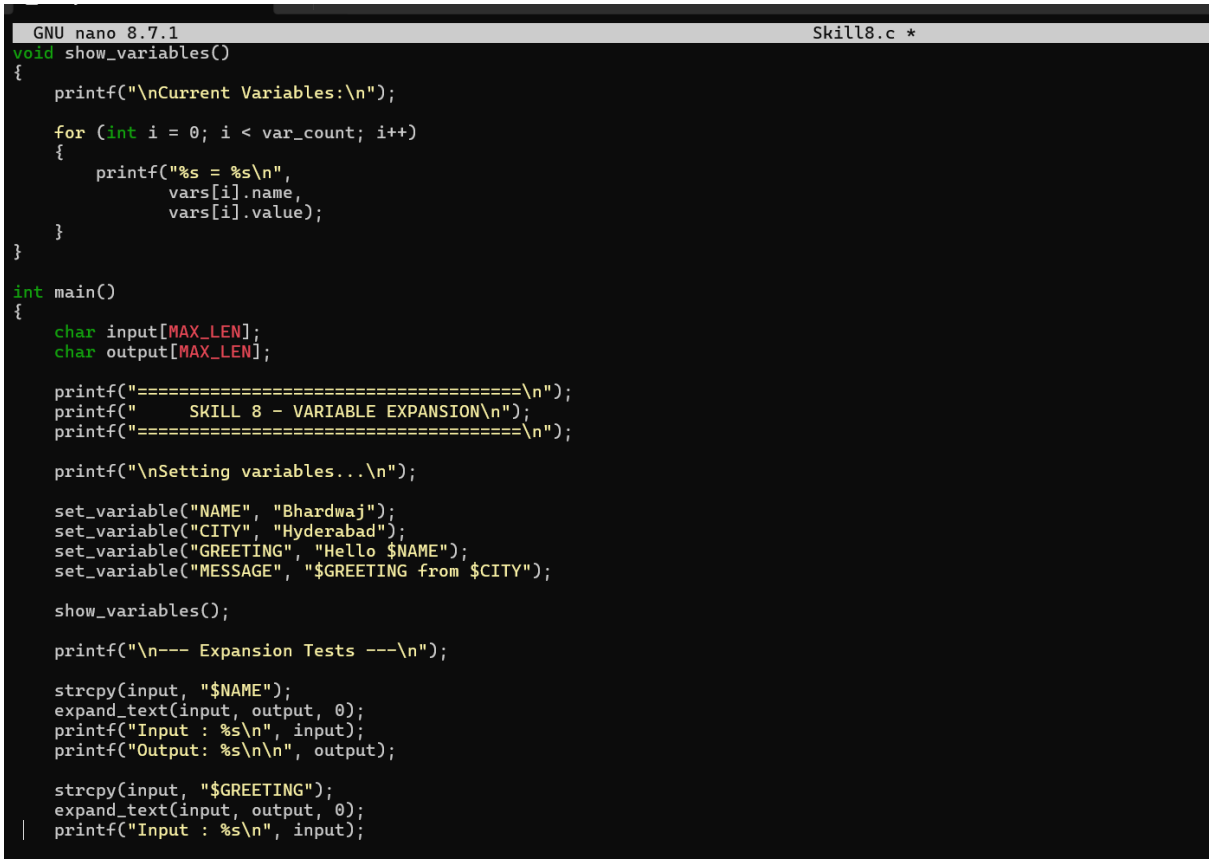
            if (j == 0)
            {
                strcat(temp, "$");
                continue;
            }

            char *value = get_variable(name);

            if (value == NULL)
            {
                strcat(temp, "[UNDEFINED]");
            }
            else
            {
                char expanded[MAX_LEN];
                expand_text(value, expanded, depth + 1);
                strcat(temp, expanded);
            }
        }
        else
        {
            int len = strlen(temp);
            temp[len] = input[i];
            temp[len + 1] = '\0';
            i++;
        }
    }

    strcpy(output, temp);
}

/* Display all variables */
void show_variables()
```



```
 GNU nano 8.7.1                                  Skill8.c *
void show_variables()
{
    printf("\nCurrent Variables:\n");

    for (int i = 0; i < var_count; i++)
    {
        printf("%s = %s\n",
               vars[i].name,
               vars[i].value);
    }
}

int main()
{
    char input[MAX_LEN];
    char output[MAX_LEN];

    printf("=====================================\n");
    printf("     SKILL 8 - VARIABLE EXPANSION\n");
    printf("=====================================\n");

    printf("\nSetting variables...\n");

    set_variable("NAME", "Bhardwaj");
    set_variable("CITY", "Hyderabad");
    set_variable("GREETING", "Hello $NAME");
    set_variable("MESSAGE", "$GREETING from $CITY");

    show_variables();

    printf("\n--- Expansion Tests ---\n");

    strcpy(input, "$NAME");
    expand_text(input, output, 0);
    printf("Input : %s\n", input);
    printf("Output: %s\n\n", output);

    strcpy(input, "$GREETING");
    expand_text(input, output, 0);
    printf("Input : %s\n", input);
```

```
 GNU nano 8.7.1                                                Skill8.c *
    printf("Input : %s\n", input);
    printf("Output: %s\n\n", output);

    strcpy(input, "$GREETING");
    expand_text(input, output, 0);
    printf("Input : %s\n", input);
    printf("Output: %s\n\n", output);

    strcpy(input, "$MESSAGE");
    expand_text(input, output, 0);
    printf("Input : %s\n", input);
    printf("Output: %s\n\n", output);

    strcpy(input, "Welcome $NAME to $CITY");
    expand_text(input, output, 0);
    printf("Input : %s\n", input);
    printf("Output: %s\n\n", output);

    strcpy(input, "Value = $UNKNOWN");
    expand_text(input, output, 0);
    printf("Input : %s\n", input);
    printf("Output: %s\n\n", output);

    printf("--- Updating Variable ---\n");

    set_variable("NAME", "Surya");
    strcpy(input, "$GREETING");
    expand_text(input, output, 0);

    printf("Updated NAME = Surya\n");
    printf("Input : %s\n", input);
    printf("Output: %s\n", output);

    printf("\n====================================\n");
    printf("Variable expansion completed.\n");
    printf("====================================\n");

    return 0;
}

^G Help          ^O Write Out     ^F Where Is      ^K Cut           ^T Execute       ^C Location      M-U Undo         M-A Set Mark     M-] To Bracket
^X Exit          ^R Read File     ^\ Replace       ^U Paste         ^J Justify       ^/ Go To Line    M-E Redo         M-6 Copy         ^B Where Was
```

Output:

```
uday@DESKTOP-47L1GTF:~/2520090089_OSSP/2520090089_Skill/Skill-08$ nano Skill8.c
uday@DESKTOP-47L1GTF:~/2520090089_OSSP/2520090089_Skill/Skill-08$ gcc Skill8.c -o Skill8
uday@DESKTOP-47L1GTF:~/2520090089_OSSP/2520090089_Skill/Skill-08$ ./Skill8
====================================
     SKILL 8 - VARIABLE EXPANSION
====================================

Setting variables...

Current Variables:
NAME = Bhardwaj
CITY = Hyderabad
GREETING = Hello $NAME
MESSAGE = $GREETING from $CITY

--- Expansion Tests ---
Input : $NAME
Output: Bhardwaj

Input : $GREETING
Output: Hello Bhardwaj

Input : $MESSAGE
Output: Hello Bhardwaj from Hyderabad

Input : Welcome $NAME to $CITY
Output: Welcome Bhardwaj to Hyderabad

Input : Value = $UNKNOWN
Output: Value = [UNDEFINED]

--- Updating Variable ---
Updated NAME = Surya
Input : $GREETING
Output: Hello Surya

====================================
Variable expansion completed.
====================================
uday@DESKTOP-47L1GTF:~/2520090089_OSSP/2520090089_Skill/Skill-08$
```

nano skill8_dispatch.c:

```
 GNU nano 8.7.1                                                skill8_dispatch.c *
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <stdlib.h>

#define MAX_INPUT 200

/* Built-in command functions */
void cmd_pwd(char *args)
{
    char path[200];

    if (getcwd(path, sizeof(path)) != NULL)
        printf("Current Directory: %s\n", path);
}

void cmd_echo(char *args)
{
    if (args != NULL)
        printf("%s\n", args);
}

void cmd_set(char *args)
{
    static char state[100] = "default";

    if (args == NULL || strlen(args) == 0)
    {
        printf("Current state: %s\n", state);
    }
    else
    {
        strcpy(state, args);
        printf("State updated to: %s\n", state);
    }
}

void cmd_help(char *args)
{
    printf("\nAvailable Built-in Commands:\n");

^G Help          ^O Write Out     ^F Where Is      ^K Cut           ^T Execute       ^C Location      M-U Undo         M-A Set Mark     M-] To Bracket
^X Exit          ^R Read File     ^\ Replace       ^U Paste         ^J Justify       ^/ Go To Line    M-E Redo         M-6 Copy         ^B Where Was
```

```
 GNU nano 8.7.1                                                skill8_dispatch.c *
    printf("\nAvailable Built-in Commands:\n");
    printf("  pwd       - Display current directory\n");
    printf("  echo      - Display a message\n");
    printf("  set       - Set or display shell state\n");
    printf("  help      - Display available commands\n");
    printf("  exit      - Exit the shell\n");
}

void cmd_exit(char *args)
{
    printf("Exiting Skill 8 command dispatcher...\n");
    exit(0);
}

/* Command structure */
struct Command
{
    char name[20];
    void (*function)(char *);
};

/* Dispatch table */
struct Command dispatch_table[] =
{
    {"pwd", cmd_pwd},
    {"echo", cmd_echo},
    {"set", cmd_set},
    {"help", cmd_help},
    {"exit", cmd_exit}
};

int command_count =
    sizeof(dispatch_table) / sizeof(dispatch_table[0]);

/* Find command in dispatch table */
int find_command(char *command)
{
    for (int i = 0; i < command_count; i++)
    {
        if (strcmp(dispatch_table[i].name, command) == 0)

^G Help          ^O Write Out     ^F Where Is      ^K Cut           ^T Execute       ^C Location      M-U Undo         M-A Set Mark     M-] To Bracket
^X Exit          ^R Read File     ^\ Replace       ^U Paste         ^J Justify       ^/ Go To Line    M-E Redo         M-6 Copy         ^B Where Was
```

```
 GNU nano 8.7.1                                                skill8_dispatch.c *
    return -1;
}

int main()
{
    char input[MAX_INPUT];

    printf("====================================\n");
    printf("      SKILL 8 - COMMAND DISPATCH\n");
    printf("====================================\n");

    printf("\nBuilt-in commands loaded: %d\n",
           command_count);

    printf("\nTesting dispatch logic...\n");

    printf("\n> help\n");
    cmd_help(NULL);

    printf("\n> pwd\n");
    cmd_pwd(NULL);

    printf("\n> echo Hello OS\n");
    cmd_echo("Hello OS");

    printf("\n> set running\n");
    cmd_set("running");

    printf("\n> set\n");
    cmd_set(NULL);

    printf("\n> unknown\n");

    if (find_command("unknown") == -1)
        printf("Error: Invalid command\n");

    printf("\n--- Interactive Command Dispatcher ---\n");
    printf("Type 'help' for commands.\n");

    while (1)

^G Help          ^O Write Out     ^F Where Is      ^K Cut           ^T Execute       ^C Location      M-U Undo         M-A Set Mark     M-] To Bracket
^X Exit          ^R Read File     ^\ Replace       ^U Paste         ^J Justify       ^/ Go To Line    M-E Redo         M-6 Copy         ^B Where Was
```

```
 GNU nano 8.7.1                                                skill8_dispatch.c *
    while (1)
    {
        printf("\nskill8> ");

        if (fgets(input, sizeof(input), stdin) == NULL)
            break;

        input[strcspn(input, "\n")] = '\0';

        if (strlen(input) == 0)
            continue;

        char command[50];
        char args[MAX_INPUT];

        command[0] = '\0';
        args[0] = '\0';

        sscanf(input, "%49s %[^\n]",
               command, args);

        int index = find_command(command);

        if (index == -1)
        {
            printf("Error: Invalid command '%s'\n",
                   command);
        }
        else
        {
            dispatch_table[index].function(
                strlen(args) > 0 ? args : NULL
            );
        }
    }

    return 0;
}

^G Help          ^O Write Out     ^F Where Is      ^K Cut           ^T Execute       ^C Location      M-U Undo         M-A Set Mark     M-] To Bracket
^X Exit          ^R Read File     ^\ Replace       ^U Paste         ^J Justify       ^/ Go To Line    M-E Redo         M-6 Copy         ^B Where Was
```

Output:

```
------------------------------------
uday@DESKTOP-47L1GTF:~/2520090089_OSSP/2520090089_Skill/Skill-08$ nano skill8_dispatch.c
uday@DESKTOP-47L1GTF:~/2520090089_OSSP/2520090089_Skill/Skill-08$ gcc skill8_dispatch.c -o skill8_dispatch
uday@DESKTOP-47L1GTF:~/2520090089_OSSP/2520090089_Skill/Skill-08$ ./skill8_dispatch
====================================
     SKILL 8 - COMMAND DISPATCH
====================================

Built-in commands loaded: 5

Testing dispatch logic...

> help

Available Built-in Commands:
  pwd       - Display current directory
  echo      - Display a message
  set       - Set or display shell state
  help      - Display available commands
  exit      - Exit the shell

> pwd
Current Directory: /home/uday/2520090089_OSSP/2520090089_Skill/Skill-08

> echo Hello OS
Hello OS

> set running
State updated to: running

> set
Current state: running

> unknown
Error: Invalid command

--- Interactive Command Dispatcher ---
Type 'help' for commands.

skill8>
```